

The Job Sequencing and Tool Switching Problem

Part VI: Grouping Selection for the JGP+GSP Heuristic

Scope and Status. This document addresses the following problem: given a pool of candidate groupings (optimal and sub-optimal solutions to the JGP), how should we select groupings to feed into the GSP so that the resulting SSP cost is minimised? We present: (i) a proved characterisation of when a fixed grouping achieves the SSP optimum via GTSP; (ii) the Max-Weight Hamiltonian Path (MWHP) result — **proved only for instances where groups at distance 2 have disjoint tool sets**; (iii) a Lagrangian relaxation framework for exploring the JGP/SSP tradeoff; (iv) ML-based grouping scoring approaches (proposed, not proved). All proved vs. proposed results are clearly labelled.

Contents

1	Setup: The Grouping Selection Problem	2
2	GTSP for a Fixed Grouping (Proved)	2
3	Max-Weight Hamiltonian Path and Tool Overlap (Partly Proved)	2
3.1	The Tool-Overlap Graph	2
3.2	The MWHP Result — with Proof Conditions	2
3.3	The Sliding-Window Instance Family	3
4	Lagrangian Relaxation: Exploring the JGP/SSP Tradeoff (Proposed)	4
5	Chromatic Structure and the Conflict Graph (Proved Observation)	4
6	Machine Learning Directions (Proposed)	4
6.1	GNN-Based Grouping Scorer	4
6.2	RL for Joint Grouping and Sequencing	5
6.3	Learning-Augmented Column Generation	5
7	Summary: What Is Proved vs. Proposed	5

Setup: The Grouping Selection Problem

After solving the JGP (via PORTA, Branch-and-Price, or a Golang enumeration), we have access to:

- $\mathcal{G}^* = \{ \text{all JGP-optimal groupings with } K^* \text{ groups} \};$
- $\mathcal{G}_> = \{ \text{sub-optimal groupings with } K > K^* \text{ groups} \}.$

For each grouping $\mathcal{G} = \{G_1, \dots, G_K\}$, let $\text{OPT}(\mathcal{G})$ denote the optimal JGP+GSP heuristic cost when restricted to configurations from the clusters $\{\mathcal{C}(G_k)\}_{k=1}^K$.

The goal is: find $\mathcal{G}^\dagger = \arg \min_{\mathcal{G} \in \mathcal{G}^* \cup \mathcal{G}_>} \text{OPT}(\mathcal{G})$.

Since $\text{OPT}(\mathcal{G})$ is itself a GTSP and expensive to compute, we need efficient *proxies* to rank groupings without solving the full GTSP each time.

GTSP for a Fixed Grouping (Proved)

Theorem 2.1 (GTSP equivalence for fixed grouping). *For a fixed grouping $\mathcal{G} = \{G_1, \dots, G_K\}$, $\text{OPT}(\mathcal{G}) = \text{min-cost Hamiltonian path on the GTSP over clusters } \{\mathcal{C}(G_k)\}_{k=1}^K \text{ with costs } d(C_i, C_j).$*

Proof. Any feasible SSP solution restricted to the grouping $\mathcal{G}$ corresponds to: selecting one configuration $C_k \in \mathcal{C}(G_k)$ per group, and choosing an ordering of the groups. This is precisely a Hamiltonian path visiting one node per cluster with arc costs $d(C_i, C_j)$, i.e., a GTSP (path variant). Minimising over both configuration choice and ordering is the GTSP. $\square$ $\square$

Practical solving. The Noon–Bean transformation [?] converts a GTSP to a standard TSP on $O(N)$ nodes ($N = \sum_k |\mathcal{C}(G_k)|$), solvable by Concorde or LKH-3. For small K^* , Held–Karp DP for GTSP runs in $O(N^2 \cdot 2^{K^*})$.

Note that $|\mathcal{C}(G_k)| = \binom{m - |T^{(k)}|}{s(G_k)}$ is exponential in the slack $s(G_k)$, so exact GTSP is tractable only for small instances or small slack. Column generation can be used to implicitly represent the cluster.

Max-Weight Hamiltonian Path and Tool Overlap (Partly Proved)

The Tool-Overlap Graph

Definition 3.1 (Tool-overlap graph). For a grouping $\mathcal{G}$, define the complete weighted graph H_{ov} on K nodes (one per group) with edge weight:

$$w(G_k, G_{k'}) = \left| \bigcup_{j \in G_k} T_j \cap \bigcup_{j \in G_{k'}} T_j \right|.$$

Under lookahead configurations (from ?? in `05_jgp_ssp_gap_analysis.tex`), the switching cost at boundary $k \rightarrow k+1$ is $r_k - w(G_k, G_{k+1})$ where $r_k = |\bigcup_{j \in G_k} T_j|$. Minimising total switching cost is therefore equivalent to maximising $\sum_k w(G_{\pi(k)}, G_{\pi(k+1)})$ over orderings π : a **Max-Weight Hamiltonian Path (MWHP)** on H_{ov} .

The MWHP Result — with Proof Conditions

Theorem 3.2 (Optimal ordering as MWHP). *For a grouping $\mathcal{G}$ where all groups have equal mandatory tool set size $r = |\bigcup_{j \in G_k} T_j|$ for all k , and where groups at distance 2 have disjoint*

mandatory tool sets (i.e., $(\bigcup_{j \in G_k} T_j) \cap (\bigcup_{j \in G_{k+2}} T_j) = \emptyset$ for all k), the optimal ordering π^* under lookahead configurations is a MWHP on H_{ov} :

$$\text{OPT}(\mathcal{G})_{\text{lookahead}} = (K-1)r - \max_{\pi} \sum_{k=1}^{K-1} w(G_{\pi(k)}, G_{\pi(k+1)}).$$

Proof. Under lookahead, $C_{\pi(k)} = T^{(\pi(k))} \cup X_{\pi(k)}$ where $X_{\pi(k)} \subseteq T^{(\pi(k+1))} \setminus T^{(\pi(k))}$ and $|X_{\pi(k)}| = s$ (the slack). Then:

$$C_{\pi(k)} \setminus C_{\pi(k+1)} = T^{(\pi(k))} \setminus C_{\pi(k+1)},$$

since $X_{\pi(k)} \subseteq T^{(\pi(k+1))} \subseteq C_{\pi(k+1)}$. Now $C_{\pi(k+1)} = T^{(\pi(k+1))} \cup X_{\pi(k+1)}$ where $X_{\pi(k+1)} \subseteq T^{(\pi(k+2))} \setminus T^{(\pi(k+1))}$. The distance-2 disjointness condition ensures $T^{(\pi(k))} \cap X_{\pi(k+1)} = \emptyset$ (since $X_{\pi(k+1)} \subseteq T^{(\pi(k+2))}$ and $T^{(\pi(k))} \cap T^{(\pi(k+2))} = \emptyset$). Therefore $T^{(\pi(k))} \cap C_{\pi(k+1)} = T^{(\pi(k))} \cap T^{(\pi(k+1))}$, and:

$$d(C_{\pi(k)}, C_{\pi(k+1)}) = |T^{(\pi(k))} \setminus C_{\pi(k+1)}| = r - |T^{(\pi(k))} \cap T^{(\pi(k+1))}| = r - w(G_{\pi(k)}, G_{\pi(k+1)}).$$

Summing over $k = 1, \dots, K-1$ and minimising over π is equivalent to maximising $\sum_k w(G_{\pi(k)}, G_{\pi(k+1)})$. $\square$ $\square$

Caveat on the distance-2 disjointness condition. The proof above uses $T^{(\pi(k))} \cap T^{(\pi(k+2))} = \emptyset$. This holds for the sliding-window construction of section 3.3 (where groups are spaced far enough apart), but does *not* hold for general arbitrary groupings. For general groupings, $T^{(\pi(k))} \cap X_{\pi(k+1)}$ may be non-zero, and the formula $d(C_{\pi(k)}, C_{\pi(k+1)}) = r - w(G_{\pi(k)}, G_{\pi(k+1)})$ no longer holds. Using the MWHP heuristic for group ordering on general instances is a reasonable approximation but is not guaranteed to find the true GSP optimum.

The Sliding-Window Instance Family

The MWHP result is exact for the following structured instance family, which also gives tight instances for Part I of `05_jgp_ssp_gap_analysis.tex`.

Definition 3.3 (Sliding-window instance $\mathcal{I}(K, r, ov, b)$). Parameters: K groups, group tool set size r , overlap ov between adjacent groups, magazine capacity b , satisfying: $0 \leq ov \leq r/2$, $r < b$, $2r - ov > b$.

- **Tools:** $T = \{t_1, \dots, t_m\}$ with $m = (K-1)(r - ov) + r$.
- **Mandatory sets:** $T^{(k)} = \{t_{(k-1)(r-ov)+1}, \dots, t_{(k-1)(r-ov)+r}\}$ (consecutive windows of width r , shifted by $r - ov$).
- **Adjacency:** $|T^{(k)} \cap T^{(k+1)}| = ov$; $|T^{(k)} \cup T^{(k+1)}| = 2r - ov > b$ (adjacent groups cannot share a configuration $\Rightarrow$ they must be in different groups).
- **Distance-2 disjointness:** By $ov \leq r/2$, the overlap region of $T^{(k)}$ and $T^{(k+1)}$ does not reach $T^{(k+2)}$, so $T^{(k)} \cap T^{(k+2)} = \emptyset$.

For this family, $K^* = K$ (no adjacent groups can be merged) and the MWHP result applies exactly. The natural linear ordering $G_1 \rightarrow G_2 \rightarrow \dots \rightarrow G_K$ maximises the overlap ov at each boundary and is the MWHP optimum. The optimal lookahead GSP cost is $(K-1)(r - ov)$.

Corollary 3.4 (Optimal SSP cost for the sliding-window family). *Combined with the mandatory*

exit lower bound (?? in `05_jgp_ssp_gap_analysis.tex`), both bounds equal $(K - 1)(r - \text{ov})$, so $Z_{\text{SSP}}^* = (K - 1)(r - \text{ov})$ for $\mathcal{I}(K, r, \text{ov}, b)$.

Practical use. For arbitrary instances, compute H_{ov} and solve MWHP as a proxy for the optimal group ordering (tractable via Held–Karp DP in $O(K^2 2^K)$, feasible for small K^*). Then run the full GTSP on the top-ranked orderings. This is a heuristic shortcut, not an exact method, for general instances.

Lagrangian Relaxation: Exploring the JGP/SSP Tradeoff (Proposed)

Status: Proposed direction, not yet proved or implemented.

The JGP minimises group count $|\mathcal{G}|$; the SSP minimises switching cost. These objectives can conflict: a grouping with $K > K^*$ groups may yield a lower switching cost than any K^* -group solution (as conjectured in `05_jgp_ssp_gap_analysis.tex`, TODO 2).

A Lagrangian relaxation penalises extra groups:

$$L(\lambda) = \min_{\mathcal{G}, \pi, \{C_k\}} \left[\text{cost}(\mathcal{G}, \pi, \{C_k\}) + \lambda(|\mathcal{G}| - K^*) \right], \quad \lambda \geq 0.$$

By weak duality, $L(\lambda) \leq Z_{\text{SSP}}^*$ for all $\lambda \geq 0$. As λ decreases from ∞ to 0:

- Large λ : penalises extra groups heavily $\Rightarrow$ optimal at K^* groups (same as JGP).
- Small λ : groups are cheap $\Rightarrow$ more groups, lower switching cost per group.
- $\lambda = 0$: pure SSP minimisation.

The subgradient algorithm traces a Pareto frontier of (group count, switching cost), providing sub-optimal groupings at various $K > K^*$ that may achieve $\text{cost}(\mathcal{G}, \pi^*) < \text{OPT}(\mathcal{G}^*)$ for any K^* -group solution. The tightest lower bound is $L(\lambda^*) = \max_{\lambda \geq 0} L(\lambda)$; the gap $Z_{\text{SSP}}^* - L(\lambda^*)$ equals the LP integrality gap.

Chromatic Structure and the Conflict Graph (Proved Observation)

Definition 5.1 (Job conflict graph [?]). $G_c = (J, E_c)$ where $(i, j) \in E_c \iff |T_i \cup T_j| > b$ (jobs i and j cannot share a configuration).

Valid JGP groupings correspond to proper colourings of G_c : each colour class is a feasible group. The chromatic number $\chi(G_c) = K^*$.

Remark 5.2 (MWHP connects chromatic structure to switching cost). Among all K^* -colourings of G_c , the MWHP criterion (section 3) selects the colouring whose colour classes have maximum pairwise mandatory tool overlap in the optimal ordering. This connects the chromatic structure of G_c to the SSP objective. To our knowledge, this connection is not made explicit in prior literature.

Machine Learning Directions (Proposed)

Status: Proposed directions. None are proved or implemented yet. Mathematical justifications are given where available.

GNN-Based Grouping Scorer

Goal. Score a large pool of candidate groupings (from PORTA/Golang enumeration) without solving the GTSP for each.

Architecture. Encode the SSP instance as a bipartite job–tool graph $B = (J \cup T, E)$ with $(j, t) \in E \iff t \in T_j$. A two-layer Graph Attention Network (GAT) produces job embeddings $h_j \in \mathbb{R}^d$. For grouping $\mathcal{G}$, represent group G_k as the mean embedding $\mathbf{g}_k = |G_k|^{-1} \sum_{j \in G_k} h_j$. A second GNN on the K -node overlap graph H_{ov} (edge features $w(G_k, G_{k'})$) predicts $\text{OPT}(\mathcal{G})$.

Mathematical justification. $\text{OPT}(\mathcal{G})$ depends on the pairwise tool overlap between groups, which is captured by H_{ov} . GNNs are permutation-equivariant and can approximate any function on graphs [?]; since SSP cost is permutation-invariant in the group labels, the architecture is principled.

Training. Solve instances exactly up to moderate size; generate both $K = K^*$ and $K > K^*$ groupings; train on exact $\text{OPT}(\mathcal{G})$ values.

Guarantee. If the GNN approximates $\text{OPT}(\mathcal{G})$ within error ε with probability $\geq 1 - \delta$, the returned best grouping has cost $\leq Z_{\text{SSP}}^* + \varepsilon + \eta$ where $\eta \rightarrow 0$ as the number of evaluated groupings grows.

RL for Joint Grouping and Sequencing

Goal. Learn a policy that directly constructs groupings to minimise switching cost, without being constrained to K^* groups.

MDP. State: partial job-to-group assignment. Action: assign the next unassigned job to an existing feasible group or open a new one (masked to satisfy $|\bigcup_{j \in G} T_j| \leq b$). Terminal reward: $R_T = -\text{cost}(\mathcal{G}, \pi^*, \{C_k^*\})$ (negative GSP cost, computed after the episode ends).

Training. REINFORCE with greedy rollout baseline [?]: $\nabla_{\theta} \mathcal{L} = -\mathbb{E}[(R_T - b) \nabla_{\theta} \log \pi_{\theta}]$.

Key distinction. A JGP solver minimises group count. This RL policy minimises switching cost directly, and may discover that $K > K^*$ groups are beneficial — empirically testing the conjecture of TODO 2 in `05-jgp-ssp-gap-analysis.tex`.

Guarantee. No worst-case bound; distributional convergence only.

Learning-Augmented Column Generation

Setup. The JGP is solved via column generation [?]. The pricing subproblem (finding a maximal feasible group with minimum reduced cost) is itself NP-hard. An ML predictor $P_{\phi} : (u, B) \rightarrow G'$ (where u are dual variables and B is the instance graph) acts as a fast heuristic oracle.

Protocol. At each CG iteration: query P_{ϕ} ; check reduced cost of G' (cheap); if negative, add to master; else fall back to exact pricing. All ML-suggested columns are verified before use; LP optimality is certified by the fallback. Exact LP optimality is preserved; ML only reduces pricing calls. Analogous approaches reduce pricing iterations by 50–80% on related problems [?].

Summary: What Is Proved vs. Proposed

Result	Status	Location
GTSP for fixed grouping	Proved	theorem 2.1
MWHP = optimal ordering (distance-2 disjoint instances)	Proved	theorem 3.2
MWHP for general instances	Open / Approximation only	section 3
Sliding-window family: $Z_{\text{SSP}}^* = (K - 1)(r - \text{ov})$	Proved	Corollary
Lagrangian Pareto frontier	Proposed	section 4
Chromatic/MWHP connection	Proved observation	section 5
GNN grouping scorer	Proposed	section 6
RL for joint grouping	Proposed	section 6
ML-augmented CG	Proposed	section 6